WEALTH PULSE: CRITICAL PRIORITY ANALYSIS

Deliverable ID: WP-CPA-04-0001

Project Title: Wealth Pulse Cross-Asset Portfolio and Educational Suite

Date: June 18, 2026

Table of Contents

DOCUMENT METADATA

Document Ownership

Name	Position / Role	Organization	Email
Gage Radtke	Lead Software Engineer / Project Manager	Wilmington University	gradtke001@wilmu.edu
Dr. Charles Bachman	Capstone Advisor / Project Manager Reviewer	Wilmington University	cbachman@wilmu.edu

Revision History

Date	Revision	Summary of Changes	Author
06/18/2026	1.0	Initial Release: Defined System-Building vs. System-Execution Constraints and Critical Severity Risk Matrix.	G. Radtke

1. INTRODUCTORY FRAMEWORK & METHODOLOGY

The Critical Priority Analysis serves as the architectural stress-test for *Wealth Pulse*. It isolates the tasks that represent the absolute boundary between project success and project failure. If any single task designated within this analysis is executed poorly or fails to resolve at runtime, the entire application collapses from an engineering and functional perspective. Conversely, non-critical tasks (such as extended UI visual aesthetics or deep historical news archiving) can be delayed or altered without threatening the structural survival of the system.

In alignment with the structural principles of the Water Sluice software engineering methodology, setting clear priorities directly dictates how resources are used and how risks are managed throughout the project life cycle. This analysis divides the core priorities of *Wealth Pulse* into two distinct categories:

- System-Building Critical Tasks: Core development, compilation, and infrastructure efforts that the developer must execute perfectly during the construction phase on the Arch Linux environment.
- System-Execution Critical Tasks: Real-time operational behaviors that the software application itself must perform flawlessly once it is compiled, deployed, and interacting with live data streams.

2. CATEGORY 1

1: SYSTEM-BUILDING CRITICAL TASKS (DEVELOPMENT LIFECYCLE)

These are the technical implementation boundaries that must be built correctly within your Code-OSS IDE and Java 21 environment. Failure here means the application will be plagued by compilation defects, security gaps, or data corruption before it ever runs.

Task 1.1: High-Precision Relational Database Schema Mapping

- Description: Designing and migrating the physical PostgreSQL table structures to map users, traditional equities, and precious metals holdings.
- Why it is Critical: Financial ledger applications live or die based on data integrity. If asset quantities or prices are stored using imprecise floating-point numbers (FLOAT or DOUBLE), binary rounding errors will compound during portfolio calculations, resulting in inaccurate balances.
- Implementation Standard: The developer must explicitly build the database schemas and Java @Entity models using high-precision decimal mapping (DECIMAL(19,4) on PostgreSQL and BigDecimal inside JDK 21). This step ensures the software tracks fractions of alternative asset weights (such as fine troy ounces of gold or silver) with exact precision.

Task 1.2: Implementation of the Multi-Tier API Caching Service Layer

- Description: Programming the operational logic inside AssetService.java to handle data syncing between your backend, the database, and third-party endpoints (Alpha Vantage and GoldAPI.io).
- Why it is Critical: Operating on free-tier APIs introduces a strict project constraint: a maximum limit of 5 calls per minute. If the developer builds a standard direct-pass API connection, any user refreshing the React dashboard three or four times will lock out the API keys, causing the app components to break.
- Implementation Standard: The developer must build a bulletproof local caching component. The service layer must check the local PostgreSQL rate-limit cache table first. It must only trigger an outbound web call if the existing timestamp is older than 15 minutes, cleanly protecting the application from hitting free-tier blocks.

Task 1.3: Stateless Authentication and Authorization Pipeline Containment

- Description: Structuring the Spring Security filter chain container, configuring BCrypt password hashing, and writing the token verification rules inside the backend application.
- Why it is Critical: Because *Wealth Pulse* aggregates a user's private financial positions and net worth metrics, unauthorized data access constitutes a catastrophic failure. A weak security setup that allows users to view or

manipulate another account's asset rows would invalidate the project as a viable capstone.

- Implementation Standard: The developer must configure standard Spring Security filters to intercept all inbound requests. Passwords must be cryptographically salted and hashed via BCrypt before database persistence, and the system must use stateless JSON Web Tokens (JWT) verified on every single API transaction.

3. CATEGORY 2

2: SYSTEM-EXECUTION CRITICAL TASKS (OPERATIONAL RUNTIME)

These are the operational requirements that the running application must execute correctly. Once built, the system must perform these tasks flawlessly to meet its functional goals.

Task 2.1: Accurate Multi-Asset Portfolio Valuation & Mathematical Processing

- Description: The system must accurately compute total wealth, individual asset allocations, and forward-looking retirement compound projections at runtime.
- Why it is Critical: The primary value of *Wealth Pulse* is its ability to combine traditional stocks with raw physical precious metal weights into a unified portfolio view. If the running system fails to convert ounces of gold to

currency using live spot prices, or applies an incorrect interest formula during retirement projections, it fails its core purpose.

- Implementation Standard: At runtime, when stimulated by a dashboard page mount, the backend system must instantly calculate current values across all user asset arrays:

$$\text{Asset Value} = \text{Ounces/Shares} \times \text{Current Market Spot Price}$$

The system must then calculate exact asset allocations and output structured JSON payloads to feed the frontend Chart.js components without delay.

Task 2.2: Graceful Integration Handling and Third-Party API Fault Tolerance

- Description: The system must gracefully manage external API connectivity failures, network drops, or empty JSON responses from third-party data services without crashing.
- Why it is Critical: External APIs frequently experience network drops, temporary outages, or return unexpected data formats. If the running system encounters a network timeout while trying to fetch the spot price of silver and responds by throwing an unhandled internal server error, the entire user interface will freeze or crash.
- Implementation Standard: The backend integration components must be engineered with strict error-handling blocks (try-catch blocks surrounding outbound Axios or WebClient actions). If an external API fails, the system must detect the fault, gracefully fall back to the last known cached database

spot price, and send an alert to the client UI rather than interrupting the user experience.

Task 2.3: Client-Side View State and Styling Isolation

- Description: The running React application must maintain secure user sessions and enforce strict visual isolation using CSS Modules across all dashboard modules.
- Why it is Critical: In an asset-tracking app, any visual overlap or layout breakdown completely ruins the user experience. Furthermore, if the client application fails to store the security token properly during page navigation, the user will be logged out repeatedly while trying to manage their assets.
- Implementation Standard: The frontend runtime must isolate component layouts using CSS Modules to ensure design rules never leak across pages. Simultaneously, the Axios client instance must automatically read the active JWT from localStorage and inject it into request headers, keeping the user securely authenticated as they navigate between the Asset Manager and the Learning Center.

4. CRITICAL SEVERITY MATRIX

This matrix maps out the impact of a failure in any of your critical tasks, alongside the exact architectural safeguards used to prevent it:

Critical Task Target	Category	Failure Impact Result	Architectural Safeguard Mitigation
Data Precision	System-Building	Compounding	Enforce DECIMAL(19,4) types in

Critical Task Target	Category	Failure Impact Result	Architectural Safeguard Mitigation
Mapping		mathematical errors; inaccurate user portfolio balances.	PostgreSQL and BigDecimal variables across all Java calculations.
API Caching Service	System-Building	API keys get rate-limited; live market tracking freezes for all users.	Build a database-backed cache layer inside AssetService.java to enforce a 15-minute data-reuse window.
Security Configuration	System-Building	Data breaches; users can view or alter other accounts' private asset ledgers.	Enforce BCrypt password hashing and validate stateless JWT security headers on every request.
Portfolio Valuation	System-Execution	UI charts render broken shapes; financial calculation errors occur.	Write comprehensive JUnit 5 unit tests to verify the core mathematical logic before deployment.
API Fault Tolerance	System-Execution	Unhandled network errors crash the server during external web drops.	Wrap outbound connections in robust try-catch blocks that fall back to cached data during outages.
UI Isolation & Token Drift	System-Execution	Layout styles overlap and distort; users encounter random session logouts.	Lock design styles down with hashed CSS Modules and automate token injection using Axios interceptors.

Critical Priority Analysis — Current Implementation Addendum

Deliverable: WP-CPA-04-0001 **Revision:** 2.0 **Date:** August 3, 2026

This status addendum supersedes design-phase implementation descriptions where they differ from the repository.

Current Critical Priorities

Priority	Implemented safeguard	Remaining concern
User isolation and authentication	JWT filter, BCrypt password encoding, protected routes, and repository queries scoped by owner ID.	Continue integration testing for every new user-owned operation.
Financial correctness	DTO validation, explicit cost-basis rules, an asset transaction ledger, daily snapshots, and unit tests for purchase/sale rules.	Asset entity quantity and price fields remain Double. Avoid claiming universal decimal precision.
Market-data resilience	15-minute current-price cache, 24-hour historical cache, stale-value fallback for supported failures, timeouts, and per-asset error isolation.	External calls are synchronous and provider/rate-limit behavior can still delay refresh operations.
Maintainable flow	Feature-based React structure and controller/service/repository backend layers. Performance logic is split across coordinator, snapshot, metrics, and report services.	Market news and tracker integrations remain comparatively coupled to parsing/network details.
Recoverability	Single live database, **wealth_pulse**, with consolidation backups created during cleanup.	Formal scheduled database backup and restore automation is not yet part of the repository.

Corrections to Earlier Assumptions

- The frontend uses ordinary shared/feature CSS files rather than CSS Modules.
- The backend uses RestTemplate and HttpURLConnection integrations; it does not use Spring WebClient for the described calls.
- High-precision numeric mapping is applied selectively, not to every asset field.
- API limits should be treated as provider-configurable constraints rather than a permanent universal “five calls per minute” fact.
- JWT verification is performed for protected requests, while public authentication, cache-status, news, and tracker routes are explicitly configured.

Current Severity Matrix

Failure	Impact	Required verification
Cross-user access	Critical	Test visibility, update, and deletion with two accounts.
Incorrect sale/cost basis	Critical	Unit-test purchases, partial sales, complete sales, overselling, nulls, and zero values.
Provider outage	High	Verify fresh cache, stale fallback, timeout, and no-cache error behavior.
Snapshot/benchmark gap	Medium	Return an availability flag and history message; never fabricate performance history.
UI regression	Medium	Vitest component tests, production build, Playwright flows, and mobile-width UAT.